

Integrations guide — NEDS tool ecosystem

The CRM connects to two other NEDS tools — **nedsdrishti.in** (Drishti) and **socialmediadost.com** (Social Media Dost / SMDost). This guide explains what flows automatically between the three tools, what each team member sees, and what to do when something doesn't look right.

This guide is for **managers and admins**. Sales, support and accounts staff will encounter the effects of these integrations in their day-to-day work — the relevant parts are called out in their own guides.

The three-tool picture

...

CRM (source of truth)

Clients · Deals · Invoices · Payments · Tickets

▼ ▼

nedsdrishti.in socialmediadost.com

Service delivery Content production

Audits · SEO · Brief → AI content

Analytics · Posts → Approval → Posting

...

The **CRM** is where client relationships, money, and support live. Drishti is where the service delivery team runs audits, tracking and content scheduling. SMDost is where the content team creates AI-generated posts, gets client sign-off, and queues them for publishing.

Integration 1 — Client auto-provisioning when a deal is won

What it does: When a Deal in the CRM is moved to the **Won** stage, the client is automatically created in Drishti and SMDost within seconds. No manual re-entry needed.

What the team sees:

- Sales marks the deal Won → the CRM creates the client in both tools in the background.
- The client's CRM profile gains two internal IDs: **Drishti Client ID** and **SMDost Client ID** (visible in the client record's detail section).

- An **activity entry** appears on the client's CRM timeline confirming provisioning, e.g. **"Provisioned in Drishti (ID: drsh-xyz)"**.

If provisioning fails:

- Check the client's CRM activity feed — a failure note will appear there too.
 - Re-trigger by opening the Deal, moving it back to Negotiation, and then to Won again — the job is idempotent (it won't duplicate).
 - If repeated failures occur, check the server `.env` for `DRISHTI_API_URL` , `DRISHTI_SERVICE_KEY` , and `SMDOST_SERVICE_KEY` .
-

Integration 2 — Brief approved in SMDost → draft invoice in CRM

What it does: When the content team marks all content in a brief as

Approved in SMDost, the CRM automatically creates a **Draft Invoice** for that client and notifies the accounts team.

What accounts sees:

- A bell notification: `"SMDost brief approved for [Client] — draft invoice ready to price."`
- A draft invoice appears for the client in the CRM (Invoices tab on the client profile). The line item is pre-filled with the service description; the accounts team sets the amount and sends the invoice.

What the content team needs to know:

- Make sure the client's SMDost account has the correct **CRM Client ID** set (done automatically during provisioning). If it's blank, the webhook has no way to match the brief to a CRM client and the invoice won't be created.
-

Integration 3 — Approved SMDost content → Drishti for scheduling

What it does: When a brief is fully approved in SMDost, each content item (caption + media) is automatically pushed to Drishti as a **Scheduled Post**. The Drishti team then reviews and confirms before the posts go live.

What the Drishti team sees:

- New posts appear in the Drishti posts queue with status `**Scheduled / Pending Approval**`.
- Platform mapping: Instagram, Facebook, LinkedIn, Twitter/X, TikTok, and Google Business posts each land on the correct platform account.
- Default scheduled date is the **10th of the brief's month at 9 AM**. The content team can set a specific date per item in SMDost before approving.

Nothing in the CRM changes for this flow — it runs between SMDost and Drishti directly.

Integration 4 — Drishti activity → CRM client timeline

What it does: When a post is **approved, rejected, or published** in Drishti, that event is written to the CRM client's activity feed.

What the team sees:

- Open any client profile → **Activity** tab → entries like:

"Drishti: post approved — 'Diwali offer caption'"

"Drishti: post published on Instagram"

- This means the CRM has a full picture of the client relationship — sales history, support history, and service delivery milestones — without anyone copy-pasting between tools.

Note: only clients with a Drishti Client ID will receive these events. The ID is set automatically when a deal is won (Integration 1).

Integration 5 — Monthly brief auto-creation

What it does: On the **1st of every month at 7:30 AM**, the CRM automatically creates a content brief in SMDost for each active project that has a Social Media or GMB service and a linked SMDost client.

Platform defaults per service:

Service	Platforms created
Social Media	Instagram (4 posts), Facebook (4 posts)
GMB	Google Business (4 posts)

What the content team sees:

- A new brief appears in SMDost on the 1st, ready to start producing content.

No one has to remember to create it.

- The brief title follows the format: `""[Client Name] — [Service] — [Month Year]"`.

Idempotency: if the command runs twice for the same month (e.g. a server restart), it won't create duplicate briefs — it checks the CRM activity log first.

Manual trigger (backfill):

```
```bash
php artisan app:create-monthly-briefs --month=2026-07
```
```

Run this on the server via SSH if a month was missed.

Integration 6 — Client portal single sign-on (SSO)

What it does: Clients who log into the NEDS CRM portal see a **"Your NEDS Tools"** section on their dashboard with buttons to open Drishti and/or SMDost — and they land already signed in, without a separate password.

Conditions for the button to appear:

- The client's CRM profile must have a **Drishti Client ID** (auto-set on deal

won) for the Drishti button.

- The client's CRM profile must have an **SMDost Client ID** for the SMDost button.

- The client must have a user account in the respective tool (created automatically on deal won).

How it works for the client:

1. Log into the CRM portal.
2. Click **"Open Drishti Dashboard"** or **"Open Social Media Dost"**.
3. They are redirected and automatically signed into that tool.
4. The sign-in link expires after **10 minutes** — if the client reports a sign-in error, they should return to the portal and click the button again.

If a client can't sign in via SSO:

- Check that their CRM contact has a portal login (Clients → Contacts → invite if not).
- Check that the client's CRM profile has the correct Drishti/SMDost Client ID.
- Check that the client's user account exists in Drishti and is **active**.
- If issues persist, the client can log into Drishti or SMDost directly with their email and password.

Integration 7 — Drishti context link on support tickets

What it does: When a support ticket is about a client who is connected to Drishti, the ticket show page displays a blue **"Open in Drishti →"** link so the support agent can jump straight to the relevant Drishti view without searching.

The link is service-aware:

| Ticket service | Drishti link goes to |
|---------------------------------------|--------------------------------------|
| SEO or GMB | Client's audit list |
| Social Media or Performance Marketing | Client's optimization / content view |
| Any other / no service set | Client's detail page |

WhatsApp tickets: when a WhatsApp message creates a ticket and the client is linked to Drishti, the Drishti URL is also appended to the ticket description — so it's visible in wadesk.in as well.

Integration 8 — WhatsApp two-way reply (wadesk.in)

What it does: WhatsApp is now a full two-way channel through the CRM.

- **Inbound:** when a customer messages on WhatsApp, wadesk.in calls the CRM webhook, which matches the phone number to a client and opens a **Ticket** (channel = WhatsApp) — deduplicated per wadesk.in conversation, so replies in the same conversation don't create new tickets.

- **Inbound from an unknown number:** if the phone doesn't match any client, the CRM now creates a **Lead** instead (source = WhatsApp) — so a genuine new-business enquiry over WhatsApp is never silently dropped. It's also deduplicated per conversation: later messages from the same unmatched conversation are added as notes on that lead rather than creating a second one. The lead is auto-assigned to a sales rep the same way any other new lead is (see the AI features section).
- **Outbound:** when a staff member replies on that ticket in the CRM (and the reply is **not** marked "internal note"), the CRM sends the reply back through wadesk.in so the customer receives it on WhatsApp — the staffer never has to open wadesk.in or WhatsApp directly.

What the team sees:

- A ticket tagged WhatsApp behaves like any other ticket — reply in the CRM as normal.
- Internal notes (the "internal" checkbox) are never sent to the customer — use these for team-only context.
- If the client can't be matched by phone number, a **Lead** is created instead of a ticket (see above) — check the **Lead Generation** list rather than assuming the message was lost. If it's actually an existing client with an out-of-date phone number, fix the number on their client record and ask them to send another WhatsApp message so future replies open a proper ticket.

If a customer says they didn't receive a reply:

- Check the ticket wasn't marked as an internal note by mistake.
- Check server `.env` has `WADESK_API_URL` and `WADESK_SERVICE_KEY` set — without these the outbound send is silently skipped (logged as a warning, never blocks the ticket reply itself).
- wadesk.in outages never break the CRM reply — the message is just not forwarded; the staffer may need to resend once wadesk.in is back up.

Integration 9 — Drishti marketing metrics feed the monthly wins note

What it does: the CRM's AI monthly wins note (see the AI features section in the Admin/Manager guides) pulls real marketing-delivery numbers from Drishti for clients Drishti manages — posts published, audits completed, and marketing action items done for the month, fetched from Drishti's `GET /api/clients/{id}/monthly-metrics` endpoint via the same X-Service-Key pattern as the other integrations. These are the same counts Drishti's own weekly client digest already computes, just totalled over a full month instead of a week.

What the team sees: no extra step — the monthly wins note simply mentions

these numbers alongside the CRM's own (tasks/tickets/payments) when a client has a Drishti account linked. Nothing changes for clients without one.

If Drishti is unreachable: the call fails silently (logged as a warning) and the note still drafts from whatever the CRM itself knows. This integration can never block the monthly wins note from running.

Integration 10 — Meta Lead Ads webhook

Status: built, not yet configured — needs a Facebook Developer App, a registered Page, and a Page access token before it goes live. See Section 8 of the Admin guide for setup steps.

What it does: when someone submits a Facebook or Instagram Lead Ads form, Meta calls `POST /api/webhooks/meta-leads` — but that event only carries a `leadgen_id`, not the actual name/email/phone submitted. The CRM queues a job that fetches the real field data from Meta's Graph API (`GET /{leadgen_id}?access_token=...`) and creates a Lead from it (source **Meta Ads**). Deduped on `leads.meta_leadgen_id` — a redelivered webhook event never creates a second lead.

Auth (two different mechanisms, both required):

- `GET /api/webhooks/meta-leads` — Meta's one-time (and periodic) verification handshake. Authenticated by `hub.verify_token` matching `META_WEBHOOK_VERIFY_TOKEN` — no signature involved for this request.
- `POST /api/webhooks/meta-leads` — every real lead event. Authenticated by `X-Hub-Signature-256` (HMAC-SHA256 of the raw body, keyed with `META_APP_SECRET`), verified by `VerifyMetaWebhookSignature` middleware.

Field mapping: Meta's standard field names (`full_name` or `first_name` + `last_name`, `email` / `work_email`, `phone_number` / `work_phone_number`, `company_name`) map to the lead's core fields. Any other form question (custom questions the advertiser added) is preserved as a note on the lead rather than dropped. `utm_source` / `utm_medium` are set to a fixed `meta` / `paid_social`; `utm_campaign` stores the raw `ad_id` (or `form_id` if no `ad_id`) — Meta's basic leadgen response doesn't include human-readable campaign/ad names, so the Lead Source Performance report will show IDs, not names, for this channel until that's enriched with an extra Graph API call.

What the team sees: no extra step once configured — the lead appears in **Lead Generation** with source **Meta Ads**, auto-assigned and AI-scored like any other lead (Phase A applies automatically).

If the Graph API call fails (expired token, deleted lead, rate limit): logged as a warning, no lead is created, and the job retries up to 3 times (60s backoff) before giving up silently — this must never break the webhook endpoint itself, which always responds 200 immediately after queueing.

Checking integration health

All integration events leave a trace in the CRM:

| Where to look | What it shows |
|---------------------------------|--|
| Client profile → Activity tab | Provisioning success/failure, Drishti events, brief creation |
| Client profile → Invoices tab | Draft invoices created by SMDost brief approvals |
| Tickets list → WhatsApp badge | Tickets auto-created from WhatsApp conversations |
| Drishti → Posts queue | Content pushed from SMDost |
| Ticket → replies | Outbound WhatsApp replies sent via wadesk.in |
| Lead Generation → source filter | Leads auto-created from Website, WhatsApp, and Meta Ads |

If any integration stops working, the most common causes are:

1. **Server `.env` out of date** — a key (`DRISHTI_SERVICE_KEY` , `SMDOST_SERVICE_KEY` , `PORTAL_SSO_SECRET` , `WADESK_API_URL` , `WADESK_SERVICE_KEY` , `META_APP_SECRET` , `META_WEBHOOK_VERIFY_TOKEN` , `META_PAGE_ACCESS_TOKEN` , etc.) is missing or wrong.

Run `php artisan config:cache` after any `.env` change.

1. **Docker not restarted after env change on VPS** — use `docker compose up -d` (not `restart`) so the container picks up new env vars.
1. **Client has no external ID** — if the deal was won before integrations were live, the client won't have a Drishti/SMDost ID. Provision manually by going to the deal, setting it to Won again (if safe) or by asking your developer to run the provisioning job directly.